🔥 **STAGE 2 (Checkbox 1 → 5) — MASTER CHEATSHEET**

(Async Scraping System — Architecture + Thinking Model)

👉 Read this **once**, you should be able to **rebuild the system from scratch**.

🧠 SYSTEM GOAL (TOP OF MIND — ALWAYS)

> **Fast scraping without crashing or getting banned.**
>
> Speed ❌ chaos
>
> Speed ✅ control

🧱 OVERALL ARCHITECTURE (MENTAL MAP)

```
URLs
  ↓
main()  — system controller
  ├— ClientSession (1x)
  ├— Semaphore (1x)
  ├— Tasks planning
  └— asyncio.gather()
        ↓
     fetch_html() — worker
        ├— async network call
        ├— timeout handling
        ├— status check
        ├— empty HTML check
        └— return HTML or None
```

Golden rule:

> `main()` = planner
>
> `fetch_html()` = worker

1️⃣ ASYNC MENTAL MODEL (FOUNDATION)

🔑 Core idea

- **Internet wait karta hai, CPU nahi**
- Async = wait time ko waste mat hone do

🧠 Real-life

- **Sync** = call pe lage rehna ☎️
- **Async** = WhatsApp bhej ke kaam karna 💬

🧱 Concepts

- **Blocking I/O** → network, disk, DB (rukna padta hai)
- **Event loop** → traffic controller jo ready tasks chalata hai

⚠️ Mental trap

> Async ≠ fast
> Async = *possible to be fast*

2️⃣ ASYNC SYNTAX (NO CONFUSION ZONE)

🔤 Keywords (must be reflex)

```
async def func():     # ruk sakta hai
    await something  # yahan pause

asyncio.run(main())  # engine start
asyncio.gather(...)  # tasks ek saath
```

❌ Never do

- async function bina `await` call ❌
- sync mindset ke saath async loop ❌

🧠 Rule

> **Async function ko hamesha await chahiye**
> (direct ya gather ke through)

3️⃣ AIOHTTP BASICS (NETWORK LAYER)

🔑 Why `ClientSession`

- TCP connection reuse
- Faster + safer
- Server-friendly

```
async with aiohttp.ClientSession(...) as session:
```

🔑 Why `async with session.get()`

- Request open hota hai
- Proper close hota hai
- Memory / socket leak nahi hota

🧱 Worker pattern

```
async def fetch_html(session, url):
    async with session.get(url) as response:
        return await response.text()
```

🌐 Status codes (minimum)

- `200` → OK
- `403` → blocked
- `429` → too fast
- timeout → server slow

4️⃣ CONCURRENCY CONTROL (SURVIVAL SKILL)

🔑 Core truth

> **Too fast = ban**
>
> **Controlled speed = survival**

🧠 Semaphore mental model

Semaphore = **gatekeeper** 🚪

> “Itne hi log andar ja sakte hain”

```
sem = asyncio.Semaphore(3)
```

✅ Correct placement

```
async with sem:
    async with session.get(url):
```

❌ Not outside fetch
❌ Not after response

🧠 Decision guide

Situation	Concurrency
Unknown site	3–5
Sensitive site	1–2
Large site	5–10

> Server capacity ≠ bot tolerance

5️⃣ FAILURE HANDLING (REAL WORLD READY)

🔑 Core philosophy

> **Failure is data, not death**

🧩 PART A — Partial Failure

```
results = await asyncio.gather(*tasks, return_exceptions=True)
```

- Length preserved
- Failure becomes data
- Program survives

Detect failure

```
isinstance(result, Exception)
```

🧩 PART B — Timeout Handling

Why

- Server slow
- Page hangs
- Infinite wait = system dead

Correct setup

```
timeout = aiohttp.ClientTimeout(total=10)
ClientSession(timeout=timeout)
```

Handle in fetch

```
except asyncio.TimeoutError:
    return None
```

🧩 PART C — Empty HTML & Bad Responses

Empty HTML = fake success ❌

```
if not html or len(html.strip()) < 100:
    return None
```

Status classification

- `403` → skip, don't retry
- `429` → slow down (retry later)
- `200 but empty` → useless

🧠 Output contract (VERY IMPORTANT)

After Checkbox 5:

```
fetch_html() returns:
- str   → usable HTML
- None  → failed / skipped
```

❌ No exceptions leak
❌ No crashes
✅ Clean pipeline

🧱 FINAL CANONICAL PATTERN (MEMORIZE)

```python
async def fetch_html(session, url, sem):
    try:
        async with sem:
            async with session.get(url) as response:
                if response.status != 200:
                    return None

                html = await response.text()
                if not html or len(html.strip()) < 100:
                    return None

                return html

    except (asyncio.TimeoutError, aiohttp.ClientError):
        return None


async def main():
    sem = asyncio.Semaphore(3)
    timeout = aiohttp.ClientTimeout(total=10)

    async with aiohttp.ClientSession(timeout=timeout) as session:
        tasks = [fetch_html(session, url, sem) for url in urls]
        results = await asyncio.gather(*tasks, return_exceptions=True)
        return results
```